

AI Study Coach PRO

Dokumentim i plote hap pas hapi - Backend, Frontend, API Gateway dhe Testime

Ky dokument eshte pergatitur per ta kuptuar projektin ne detaje: cfare kemi krijuar, pse e kemi krijuar, si funksionon backend-i, si funksionon frontend-i, si lidhen sherbimet me API Gateway, cilat probleme i rregulluam dhe si u testua i gjithe sistemi nga fillimi deri ne fund.

Info	Pershkrim
Emri i projektit	AI Study Coach PRO - Intelligent Study Optimization Platform
Lenda	Service Oriented Architecture
Arkitektura	Service-Oriented Architecture me API Gateway
Backend	.NET 8 ASP.NET Core Web API
Frontend	React + Vite + Tailwind CSS
Gateway	Ocelot API Gateway
Database	Entity Framework Core InMemory Database
Autentikim	JWT Authentication dhe Role-Based Authorization

1. Qellimi i projektit

Qellimi i projektit ishte te ndertojme nje aplikacion real web qe ndihmon studentet te organizojne procesin e studimit, te regjistrojne lendet dhe temat, te shtojne study sessions, te shtojne quiz results, te shohin progresin dhe te marrin rekomandime inteligjente para provimit.

Projekti nuk eshte thjesht CRUD. Ai permban business logic, sepse sistemi llogarit automatikisht total study minutes, total study hours, average score, weak areas dhe ne fund gjeneron risk level dhe exam success prediction.

2. Problemi qe zgjidh aplikacioni

Shume studente nuk kane nje menyre te qarte per te matur progresin e tyre. Ata mund te studiojne disa lende, por nuk e dine sa minuta kane studiuar, cila teme eshte me e dobet, sa eshte mesatarja e rezultateve te quiz-eve dhe ku duhet te fokusohen me shume para provimit.

- Studenti nuk e di sakte sa kohe ka studiuar.
- Studenti nuk i sheh automatikisht temat e dobeta.
- Studenti nuk merr rekomandime te personalizuar.
- Studimi shpesh behet pa plan dhe pa matje te progresit.

AI Study Coach PRO e zgjidh kete duke mbledhur te dhenat e studentit dhe duke i kthyer ato ne progres, weak areas dhe recommendations.

3. Arkitektura e pergjithshme

Projekti eshte ndertuar sipas Service-Oriented Architecture. Kjo do te thote se funksionalitetet jane ndare ne sherbime te pavarura, ku secili sherbim ka detyre te qarte.

Komponenti	Pergjegjesia
------------	--------------

UserService	Regjistrim, login, JWT token, profile, role Student/Admin
SubjectService	Lendet dhe temat
StudyService	Study sessions, quiz results, progress, weak areas
RecommendationService	Risk level, exam success prediction, recommendations
ApiGateway	Hyrja qendrore qe route-on kerkesat te sherbimet
React Frontend	UI per perdoruesin dhe komunikim me Gateway

Rrjedha kryesore e komunikimit eshte:

React Frontend -> ApiGateway -> UserService / SubjectService / StudyService / RecommendationService

Frontend-i nuk komunikon direkt me secilin service. Ai komunikon me ApiGateway ne portin 5000. Pastaj Ocelot e dergon request-in te sherbimi i duhur.

4. Struktura e projektit ne folder

Projekti u krijua ne kete folder kryesor:

C:\Users\brike\Documents\AIStudyCoachPRO

Brenda ketij folderi kemi solution-in dhe te gjitha projektet backend + frontend:

```
AIStudyCoachPRO
├── AIStudyCoachPRO.sln
├── UserService
├── SubjectService
├── StudyService
├── RecommendationService
├── ApiGateway
└── AIStudyCoachFrontend
```

AIStudyCoachPRO.sln hapet ne Visual Studio. Folderi AIStudyCoachFrontend hapet ne VS Code per React frontend.

5. Portet e perdorura

Sherbimi	Porti / URL
UserService	http://localhost:5150
SubjectService	http://localhost:5151
StudyService	http://localhost:5152
RecommendationService	http://localhost:5153
ApiGateway	http://localhost:5000
React Frontend	http://localhost:5173

Keto porte u vendosen ne launchSettings.json te secilit projekt backend, qe te mos ngaterrohen dhe qe Gateway te dije ku te dergoje request-et.

6. Backend - UserService

6.1 Cfare ben UserService

UserService eshte sherbimi per autentikim dhe perdorues. Ne kete service kemi krijuar register, login, profile dhe users endpoint. Ky service gjeneron JWT token pas login-it.

6.2 Paketat e perdorura

Microsoft.EntityFrameworkCore.InMemory 8.0.18

Microsoft.AspNetCore.Authentication.JwtBearer 8.0.18

6.3 Folderat dhe file-t kryesore

Folder / File	Qellimi
Models/ApplicationUser.cs	Modeli i perdoruesit
DTOs/RegisterDto.cs	Te dhenat per register
DTOs/LoginDto.cs	Te dhenat per login
DTOs/AuthResponseDto.cs	Pergjigja pas login-it
Data/AppDbContext.cs	DbContext per InMemory database
Services/TokenService.cs	Gjeneron JWT token
Controllers/AuthController.cs	Register, Login, Profile
Controllers/UsersController.cs	Menaxhim i users, p.sh. per Admin

6.4 JWT konfigurimi

Ne appsettings.json vendosem konfigurimin per JWT. Key, Issuer dhe Audience perdoren per te gjeneruar dhe verifikuar token-in.

```
"Jwt": {  
  "Key": "ThisIsAVeryStrongSecretKeyForAIStudyCoachPRO12345",  
  "Issuer": "AIStudyCoachPRO",  
  "Audience": "AIStudyCoachPROUsers"  
}
```

6.5 Program.cs

Ne Program.cs shtuam Authentication dhe Authorization. Gjithashtu aktivizuem Swagger me Bearer token qe te mund te testojme endpoint-et e sigurta.

- AddDbContext me InMemory Database
- TokenService si Scoped service
- JWT Bearer authentication
- Authorization
- Swagger security definition per Bearer token

6.6 Endpoint-et e UserService

Endpoint	Metoda	Qellimi
----------	--------	---------

/api/Auth/register	POST	Regjistron user te ri
/api/Auth/login	POST	Kthen JWT token dhe te dhena te user-it
/api/Auth/profile	GET	Kthen profilin e user-it te kycur
/api/Users	GET	Kthen users - endpoint per Admin

6.7 Testimi i UserService

Testuam register dhe login ne Swagger. Fillimisht krijuam user me role Student/Admin, pastaj beme login dhe morem token. Token-in e vendosem te Swagger Authorize si Bearer token.

```
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...
```

Pastaj testuam GET /api/Auth/profile dhe sistemi ktheu te dhenat e perdoruesit. Kjo vertetoi se JWT funksionon.

7. Backend - SubjectService

7.1 Cfare ben SubjectService

SubjectService menaxhon lendet dhe temat. Nje subject mund te kete disa topics. Kjo eshte lidhje one-to-many: Subject -> Topics.

7.2 Paketat

```
Microsoft.EntityFrameworkCore.InMemory 8.0.18
```

7.3 Modelet kryesore

Modeli Subject permбан Id, UserId, Name, Description, ExamDeadline, Priority dhe listen e Topics.

```
public class Subject
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public DateTime ExamDeadline { get; set; }
    public string Priority { get; set; } = "Medium";
    public List<Topic> Topics { get; set; } = new List<Topic>();
}
```

Modeli Topic permбан Id, SubjectId, Title, Difficulty, Status dhe Subject navigation property. Me vone e rregulluam me JsonIgnore per te mos krijuar object cycle.

```
using System.Text.Json.Serialization;

public class Topic
{
    public int Id { get; set; }
    public int SubjectId { get; set; }
```

```

public string Title { get; set; } = string.Empty;
public string Difficulty { get; set; } = "Medium";
public string Status { get; set; } = "Not Started";

[JsonIgnore]
public Subject? Subject { get; set; }
}

```

7.4 DTO-t

DTO	Qellimi
CreateSubjectDto	Te dhenat qe frontend dergon per subject te ri
CreateTopicDto	Te dhenat qe frontend dergon per topic te ri

7.5 Controllers

Controller	Qellimi
SubjectsController	Krijon dhe kthen subjects
TopicsController	Krijon dhe kthen topics

7.6 Endpoint-et

Endpoint	Metoda	Qellimi
/api/Subjects	GET	Kthen te gjitha lendet
/api/Subjects	POST	Krijon lende te re
/api/Topics	GET	Kthen te gjitha temat
/api/Topics	POST	Krijon teme te re
/api/Topics/subject/{subjectId}	GET	Kthen temat sipas lendes

7.7 Problemi qe e hasem: JSON cycle

Kur provuam te merrnim subjects me topics, backend-i dha error: A possible object cycle was detected. Kjo ndodhi sepse Subject kishte Topics, ndersa Topic kishte Subject, dhe JSON serializer mundohej te kthente pa fund Subject -> Topics -> Subject -> Topics.

Subject -> Topics -> Subject -> Topics -> Subject ...

Zgjidhja ishte te shtojme [JsonIgnore] te property Subject ne modelin Topic. Keshtu JSON kthehet pa u futur ne loop.

7.8 Testimi i SubjectService

Testuam fillimisht me Gateway:

<http://localhost:5000/gateway/subjects/>

Kur backend-i u restartua, u shfaq [] sepse InMemory database ishte bosh. Pastaj krijuam subject dhe topic nga frontend dhe kontrolluam prape endpoint-in. Ai ktheu subject-in me topic brenda, qe do te thote se SubjectService punonte sakte.

8. Backend - StudyService

8.1 Cfare ben StudyService

StudyService ruan study sessions dhe quiz results. Pastaj i perdor keto te dhena per te llogaritur progresin e studentit.

8.2 Paketat

Microsoft.EntityFrameworkCore.InMemory 8.0.18

8.3 Modelet

StudySession ruan sa minuta ka studiuar studenti per nje subject dhe topic.

```
public class StudySession
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public int SubjectId { get; set; }
    public int TopicId { get; set; }
    public int DurationMinutes { get; set; }
    public DateTime StudyDate { get; set; } = DateTime.UtcNow;
    public string Notes { get; set; } = string.Empty;
}
```

QuizResult ruan rezultatin e quiz-it per subject dhe topic.

```
public class QuizResult
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public int SubjectId { get; set; }
    public int TopicId { get; set; }
    public int Score { get; set; }
    public int TotalQuestions { get; set; }
    public DateTime CompletedAt { get; set; } = DateTime.UtcNow;
}
```

8.4 Controllers

Controller	Qellimi
StudySessionsController	Shton dhe kthen study sessions
QuizResultsController	Shton dhe kthen quiz results
ProgressController	Llogarit progress summary dhe weak areas

8.5 Endpoint-et

Endpoint	Metoda	Qellimi
/api/StudySessions	POST	Krijon study session

/api/StudySessions	GET	Kthen study sessions
/api/QuizResults	POST	Krijon quiz result
/api/QuizResults	GET	Kthen quiz results
/api/Progress/user/{userId}	GET	Kthen progress per user

8.6 Business logic ne ProgressController

ProgressController eshte nje nga pjeset me te rendesishme te projektit sepse ben llogaritje reale. Ai merr study sessions dhe quiz results per user-in dhe llogarit:

- totalStudyMinutes
- totalStudyHours
- totalStudySessions
- totalQuizzes
- averageScorePercentage
- weakAreas

8.7 Shembull i llogaritjes

Ne testim final kishim dy study sessions:

- Database Systems - 60 minutes
- Software Engineering - 45 minutes

Sistemi llogariti:

$60 + 45 = 105$ minutes

$105 / 60 = 1.75$ hours

Kishim dy quiz results:

- $4/10 = 40\%$
- $7/10 = 70\%$

Sistemi llogariti average score:

$40 + 70 = 110$

$110 / 2 = 55\%$

Weak area u shfaq vetem tema me 40%, sepse ishte rezultat i ulet. Tema me 70% nuk u shfaq si weak area.

9. Backend - RecommendationService

9.1 Cfare ben RecommendationService

RecommendationService analizon progresin e studentit dhe kthen Risk Level, Exam Success Prediction dhe rekomandime te personalizuar.

9.2 DTO-t

DTO	Fushat kryesore
RecommendationRequestDto	UserId, TotalStudyMinutes, AverageScorePercentage, DaysUntilExam, WeakAreasCount
RecommendationResponseDto	UserId, RiskLevel, ExamSuccessPrediction,

	Recommendations
--	-----------------

9.3 RecommendationEngine

RecommendationEngine perdor rule-based logic. Kjo do te thote se nuk perdor model AI te jashtem, por perdor rregulla te qarta per te vendosur nese studenti ka risk Low, Medium ose High.

Shembull logjike:

- Nese AverageScore < 50, shtohen risk points dhe rekomandohet fokus ne weak topics.
- Nese DaysUntilExam eshte shume afer, risk level rritet.
- Nese TotalStudyMinutes eshte i ulet, sistemi rekomandon me shume kohe studimi.
- Nese WeakAreasCount eshte me i madh se 0, sistemi rekomandon perseritje te temave te dobeta.

9.4 Endpoint

Endpoint	Metoda	Qellimi
/api/Recommendations/analyze	POST	Gjeneron recommendation per user

9.5 Testim me shembull

Testuam RecommendationService me keto te dhena:

```
{
  "userId": 2,
  "totalStudyMinutes": 60,
  "averageScorePercentage": 40,
  "daysUntilExam": 5,
  "weakAreasCount": 1
}
```

Sistemi ktheu RiskLevel High dhe ExamSuccessPrediction Low. Me vone kur kishim dy quiz-e dhe average 55%, rekomandimi u bazua ne progresin e ri.

10. ApiGateway me Ocelot

10.1 Pse e perdorem ApiGateway

Ne SOA, frontend-i nuk duhet te therrase secilin service direkt me porte te ndryshme. Per kete perdorem ApiGateway. Frontend therrret vetem http://localhost:5000/gateway dhe Gateway e dergon request-in te service i duhur.

10.2 Paketa

Ocelot 23.4.3

10.3 Program.cs i Gateway

Ne ApiGateway vendosem Ocelot, Swagger dhe CORS. CORS ishte i rendesishem sepse React frontend ne portin 5173 duhej te komunikonte me Gateway ne portin 5000.

- builder.Configuration.AddJsonFile("ocelot.json")
- builder.Services.AddCors(...)
- builder.Services.AddOcelot(...)

- `app.UseCors("AllowFrontend")`
- `await app.UseOcelot()`

10.4 ocelot.json

Ne ocelot.json vendosem routes per secilin service. Shembull:

```
Upstream: /gateway/subjects/{everything}
Downstream: /api/Subjects/{everything}
Host:     localhost
Port:     5151
```

Kjo do te thote se kur frontend therrret `/gateway/subjects/`, Gateway e dergon te `SubjectService` ne portin 5151.

10.5 Routes kryesore

Gateway route	Service
<code>/gateway/auth/{everything}</code>	<code>UserService</code>
<code>/gateway/users/{everything}</code>	<code>UserService</code>
<code>/gateway/subjects/{everything}</code>	<code>SubjectService</code>
<code>/gateway/topics/{everything}</code>	<code>SubjectService</code>
<code>/gateway/study-sessions/{everything}</code>	<code>StudyService</code>
<code>/gateway/quiz-results/{everything}</code>	<code>StudyService</code>
<code>/gateway/progress/{everything}</code>	<code>StudyService</code>
<code>/gateway/recommendations/{everything}</code>	<code>RecommendationService</code>

10.6 Problemi CORS

Ne fillim frontend-i dha Register failed sepse browser-i nuk lejonte request nga `localhost:5173` te `localhost:5000`. E rregulluam duke shtuar CORS policy ne `ApiGateway`:

```
.WithOrigins("http://localhost:5173")
.AllowAnyHeader()
.AllowAnyMethod();
```

Pas kesaj Register funksionoi me sukses nga frontend.

11. Visual Studio - Multiple Startup Projects

Qe projekti te punoje, duhet te startohen te gjitha backend service-at njeheresh. Ne Visual Studio vendosem multiple startup projects me Action = Start per:

- `UserService`
- `SubjectService`
- `StudyService`
- `RecommendationService`
- `ApiGateway`

Kur shtypim Run ne Visual Studio, hapen te gjitha sherbimet ne portet e tyre.

Nese Visual Studio thote 0 succeeded, 0 failed, 5 up-to-date, kjo nuk eshte problem. Do te thote se projektet jane te ndertuara dhe nuk ka ndryshime te reja per build.

12. Frontend - krijimi i React app

12.1 Krijimi i frontend-it

Frontend u krijua me Vite React ne folderin AIStudyCoachFrontend.

```
npm create vite@latest AIStudyCoachFrontend -- --template react
```

Pastaj u instaluan paketat:

```
npm install  
npm install axios react-router-dom tailwindcss @tailwindcss/vite
```

12.2 Tailwind setup

Ne vite.config.js shtuam Tailwind plugin:

```
import { defineConfig } from 'vite'  
import react from '@vitejs/plugin-react'  
import tailwindcss from '@tailwindcss/vite'  
  
export default defineConfig({  
  plugins: [react(), tailwindcss()],  
})
```

Ne src/index.css vendosem:

```
@import "tailwindcss";
```

13. Frontend - api.js

Krijuam file src/api/api.js per te centralizuar komunikimin me backend-in. Base URL eshte ApiGateway.

```
const api = axios.create({  
  baseURL: "http://localhost:5000/gateway",  
});
```

Shtuam interceptor qe token-i nga localStorage te dergohet automatikisht ne Authorization header:

```
api.interceptors.request.use((config) => {  
  const token = localStorage.getItem("token");  
  if (token) {  
    config.headers.Authorization = `Bearer ${token}`;  
  }  
  return config;  
});
```

Kjo do te thote se cdo request nga frontend mund te perdore token-in e user-it pas login-it.

14. Frontend - App.jsx dhe Routing

Ne App.jsx vendosem navigation bar dhe routes per faqet kryesore: Dashboard, Subjects, Study, Recommendations, Login dhe Register.

Route	Faqja
-------	-------

/	Dashboard
/subjects	Subjects
/study	Study
/recommendations	Recommendations
/login	Login
/register	Register

Me vone shtuam Logout ne navbar. Kur user klikon Logout, fshihen token, userId, fullName, email dhe role nga localStorage, pastaj user dergohet te /login.

15. Frontend - Register Page

15.1 Cfare ben Register

Register page lejon krijimin e nje user-i te ri. Form ka fullName, email, password dhe role. Kur klikohet Register, frontend dergon POST request te /auth/register permes ApiGateway.

```
api.post("/auth/register", formData)
```

15.2 Testimi

Testuam me user te ri:

Full Name: Student Frontend

Email: student20@test.com

Password: Student123!

Role: Student

Pas rregullimit te CORS, Register u krye me sukses dhe user mund te bente login.

16. Frontend - Login Page

16.1 Cfare ben Login

Login page dergon email dhe password te /auth/login. Nese login eshte i sakte, backend kthen token dhe te dhena te user-it.

```
api.post("/auth/login", formData)
```

Frontend i ruan keto ne localStorage:

- token
- userId
- fullName
- email
- role

16.2 Pse eshte e rendesishme

Keto te dhena perdoren ne Dashboard, ne navbar dhe ne request-et e ardhshme. UserId perdoret ne Study dhe Recommendations per te marre progresin e user-it te sakte.

17. Frontend - Dashboard Page

Dashboard shfaq nje pamje te pergjithshme te user-it dhe projektit. Aty tregohen emri, email, role dhe kartat me sherbimet e projektit.

- UserService - Authentication & users
- SubjectService - Subjects & topics
- StudyService - Sessions & progress
- RecommendationService - AI recommendations
- ApiGateway - Central entry point

Dashboard eshte i rendesishem per prezantim sepse tregon qarte arkitekturen dhe sherbimet.

18. Frontend - Subjects Page

18.1 Cfare ben Subjects page

Subjects page eshte per krijimin e lendeve dhe temave. Ne fillim faqja ben loadSubjects dhe loadTopics nga Gateway.

```
api.get("/subjects/")
api.get("/topics/")
```

18.2 Krijimi i subject

Forma per subject ka:

- Subject Name
- Description
- Exam Deadline
- Priority

Kur klikohet Create Subject, frontend dergon:

```
api.post("/subjects/", subjectData)
```

Pastaj e shton subject-in direkt ne state me setSubjects, qe te shfaqet pa pasur nevojë për refresh.

18.3 Krijimi i topic

Forma per topic ka:

- Subject dropdown
- Topic Title
- Difficulty
- Status

Kur klikohet Create Topic, frontend dergon:

```
api.post("/topics/", topicData)
```

18.4 Problemi qe hasem

Fillimisht subject nuk po shfaqej ne Study dhe kur ktheheshim te Subjects dukej sikur nuk ruhej. Pas kontrollit e kuptuam se backend jepte JSON cycle error. Pasi shtuam [JsonIgnore] te Topic.Subject dhe restartuam services, endpoint /gateway/subjects/ filloi te ktheje te dhena sakte.

19. Frontend - Study Page

19.1 Cfare ben Study page

Study page eshte nje nga pjeset kryesore te projektit. Ajo lejon te shtohen study sessions dhe quiz results. Gjithashtu shfaq Progress Summary.

19.2 Load data

Kur hapet Study page, ajo ngarkon:

```
api.get("/subjects/")
api.get("/topics/")
api.get("/progress/user/${userId}")
```

Prandaj nese subjects nuk ekzistojne ne backend, dropdown-i del bosh. Kur krijuam subject/topic dhe regulluam JSON cycle, Study page filloi te shfaq subjects dhe topics.

19.3 Add Study Session

Per study session vendosem:

- Subject
- Topic
- Duration Minutes
- Notes

Shembull:

```
Subject: Database Systems
Topic: SQL Joins
Duration Minutes: 60
Notes: Studied SQL joins and relational database concepts.
```

19.4 Add Quiz Result

Per quiz result vendosem:

- Subject
- Topic
- Score
- Total Questions

Shembull:

```
Subject: Database Systems
Topic: SQL Joins
Score: 4
Total Questions: 10
```

19.5 Progress Summary

Pas shtimit te study sessions dhe quiz results, Progress Summary u perditesua automatikisht dhe tregoi study minutes, study hours, quizzes, average score dhe weak areas.

20. Frontend - Recommendations Page

20.1 Cfare ben Recommendations

Recommendations page merr progress nga StudyService dhe pastaj dergon ato te RecommendationService.

Kur user klikon Generate Recommendation, ndodh kjo:

1. Frontend merr progress me GET /progress/user/{userId}.
2. Frontend krijon requestData me totalStudyMinutes, averageScorePercentage, daysUntilExam dhe weakAreasCount.
3. Frontend dergon POST /recommendations/analyze.
4. RecommendationService kthen RiskLevel, ExamSuccessPrediction dhe rekomandime.

20.2 Input

```
{
  userId,
  totalStudyMinutes: progress.totalStudyMinutes,
  averageScorePercentage: progress.averageScorePercentage,
  daysUntilExam: Number(daysUntilExam),
  weakAreasCount: progress.weakAreas.length
}
```

20.3 Output

Faqja shfaq:

- Risk Level
- Exam Success Prediction
- Personalized Recommendations

Kjo e perfundon rrjedhen kryesore te aplikacionit.

21. Frontend - Logout

Ne App.jsx shtuam logout function. Kur klikohet butoni Logout, fshihen te gjitha te dhenat nga localStorage:

- token
- userId
- fullName
- email
- role

Pastaj user dergohet te /login. Testuam dhe logout punoi me sukses.

22. Testimi final hap pas hapi

22.1 Rrjedha finale

Testimi final u be nga fillimi deri ne fund ne kete rend:

5. Register user te ri.
6. Login me user-in e regjistruar.
7. Create Subject.
8. Create Topic.

9. Add Study Session.
10. Add Quiz Result.
11. View Progress Summary.
12. Check Weak Areas.
13. Generate Recommendation.
14. Logout.

22.2 Test data - Subject 1

Field	Value
Subject Name	Database Systems
Description	Relational databases, SQL, indexing, transactions and normalization.
Exam Deadline	20 June 2026, 10:00
Priority	High
Topic	SQL Joins
Difficulty	Medium
Status	Not Started
Study Session	60 minutes
Quiz Result	4 / 10 = 40%

22.3 Test data - Subject 2

Field	Value
Subject Name	Software Engineering
Description	Software development life cycle, requirements analysis, design patterns, testing and project management.
Exam Deadline	25 June 2026, 10:00
Priority	Medium
Topic	Software Testing
Difficulty	Medium
Status	Not Started
Study Session	45 minutes
Quiz Result	7 / 10 = 70%

22.4 Rezultati final ne Progress Summary

Metric	Value	Pse eshte keshtu
Study Minutes	105	$60 + 45 = 105$
Study Hours	1.75	$105 / 60 = 1.75$
Quizzes	2	U shtuan 2 quiz results
Average Score	55%	$(40 + 70) / 2 = 55$
Weak Areas	1	Vetem 40% eshte rezultat i dobet

--	--	--

22.5 Pse 70% nuk del weak area

Sistemi tregon vetem temat ku rezultati eshte i dobet. SQL Joins me 40% u shfaq si weak area. Software Testing me 70% nuk u shfaq sepse eshte rezultat me i mire.

23. Testimi i ApiGateway

ApiGateway u testua duke hapur links direkt ne browser dhe Swagger/Postman. Shembull:

`http://localhost:5000/gateway/subjects/`

Kur database ishte bosh, ky endpoint ktheu [] dhe kjo tregoi se Gateway dhe SubjectService po punonin. Pastaj pas krijimit te subject, endpoint ktheu subject me topics.

Gjithashtu u testuan:

- `/gateway/topics/`
- `/gateway/progress/user/2`
- `/gateway/recommendations/analyze`

24. Cfare duhet te dish per InMemory Database

Projekti perdor Entity Framework Core InMemory Database. Kjo eshte e pershtatshme per testim dhe demonstrim, por ka nje sjellje te rendesishme: te dhenat fshihen kur backend services restartohen.

Prandaj kur e ndalem dhe e startuam backend-in, `/gateway/subjects/` ktheu [] sepse database u boshatis. Kjo nuk eshte bug, eshte normale per InMemory Database.

Ne te ardhmen mund te perdoret SQL Server ose PostgreSQL per ruajtje permanente te te dhenave.

25. Problemet qe i rregulluam

Problemi	Shkaku	Zgjidhja
Register failed nga frontend	CORS midis localhost:5173 dhe localhost:5000	Shtuam CORS policy ne ApiGateway
Subjects nuk shfaqeshin ne Study	Backend kthente JSON cycle error	Shtuam [JsonIgnore] te Topic.Subject
Gateway kthente []	InMemory DB ishte bosh pas restart	Krijuam subject/topic prape
Topic ruhej por subject dukej se nuk ruhej	GET subjects deshtonte per shkak te serialization cycle	Rregulluam modelin Topic
Dropdown ne Study ishte bosh	Nuk kishte subjects te kthyer nga backend	Rregulluam SubjectService dhe krijuam data

26. Si startohet projekti

26.1 Backend

15. Hap Visual Studio.

16. Hap solution: C:\Users\brike\Documents\AIStudyCoachPRO\AIStudyCoachPRO.sln
17. Sigurohu qe Multiple Startup Projects jane Start per UserService, SubjectService, StudyService, RecommendationService dhe ApiGateway.
18. Klike Run.
19. Kontrolllo Gateway: <http://localhost:5000/gateway/subjects/>

26.2 Frontend

20. Hap VS Code.
21. Hap folderin: C:\Users\brike\Documents\AIStudyCoachPRO\AIStudyCoachFrontend
22. Hap terminal.
23. Ekzekuto npm install nese node_modules nuk ekziston.
24. Ekzekuto npm run dev.
25. Hap <http://localhost:5173>

```
cd C:\Users\brike\Documents\AIStudyCoachPRO\AIStudyCoachFrontend
npm install
npm run dev
```

27. Cfare te tregosh ne prezantim

Ne prezantim duhet te shpjegosh qarte keto pika:

- Projekti quhet AI Study Coach PRO dhe ndihmon studentet te organizojne studimin.
- Eshte ndertuar me SOA sepse ka disa sherbime te ndara.
- Frontend komunikon vetem me ApiGateway.
- UserService merret me login/register dhe JWT.
- SubjectService merret me subjects/topics.
- StudyService llogarit progress dhe weak areas.
- RecommendationService gjeneron risk level dhe recommendations.
- Testimi final tregon 105 minutes, 1.75 hours, 2 quizzes dhe average 55%.
- Weak area u shfaq vetem per quiz-in me 40%.
- Logout funksionon dhe fshin localStorage.

28. Pikat qe lidhen me kerkesat e lendes

Kerkesa	Ku eshte realizuar
.NET Core Web API	UserService, SubjectService, StudyService, RecommendationService
Simple CRUD	Subjects, Topics, StudySessions, QuizResults
Business Logic	Progress calculation dhe recommendations
Complex Logic	Weak area detection dhe risk analysis
Authentication	JWT login/register ne UserService
Authorization	Roles Student/Admin dhe secured endpoints
Frontend	React + Tailwind me pages kryesore
API Gateway	Ocelot routes per secilin service
README	Duhet te vendoset ne folderin final me run instructions

Nese profesoresha pyet per Repository Layer ose Unit Tests, pergjigja e sinqerte eshte se versioni aktual fokuson SOA services, authentication, gateway, frontend dhe business logic. Repository layer dhe unit tests jane future improvement nese nuk jane implementuar plotesisht.

29. README qe duhet te vendoset ne folder

Ne folderin final mund te vendoset nje README.txt me kete permbajtje:

AI Study Coach PRO - Run Instructions

Project technology:

- .NET 8 ASP.NET Core Web API
- React
- Tailwind CSS
- Ocelot API Gateway
- JWT Authentication
- Entity Framework Core InMemory Database

How to run backend:

1. Open AIStudyCoachPRO/AIStudyCoachPRO.sln in Visual Studio.
2. Set these projects as startup projects:
 - UserService
 - SubjectService
 - StudyService
 - RecommendationService
 - ApiGateway
3. Run the backend services.

Backend ports:

- UserService: <http://localhost:5150>
- SubjectService: <http://localhost:5151>
- StudyService: <http://localhost:5152>
- RecommendationService: <http://localhost:5153>
- ApiGateway: <http://localhost:5000>

How to run frontend:

1. Open AIStudyCoachPRO/AIStudyCoachFrontend in VS Code.
2. Run npm install.
3. Run npm run dev.
4. Open <http://localhost:5173>

Important note:

The project uses Entity Framework Core InMemory Database.

Data will be cleared when backend services are restarted.

30. Folder final per dorezim

Nese te kerkohet nje folder i vetem, struktura e thjeshte mund te jete:

```
AltinIsmaili_final_project
```

```
├── AIStudyCoachPRO
```

```
├── AI_Study_Coach_PRO_Final_Paper.docx
```

```
├── AI_Study_Coach_PRO_Presentation.pptx
```

```
├── AI_Study_Coach_PRO_Demo.mp4
```

```
└── README.txt
```

Brenda AIStudyCoachPRO duhet te jete solution file, backend services dhe frontend folder.

Para ZIP-it rekomandohet te fshihet node_modules nga frontend qe file te mos behet shume i madh. Profesoresha mund ta rikrijoje me npm install.

31. Perfundim

Ne kete projekt kemi ndertuar nje aplikacion funksional me SOA. Backend-i eshte i ndare ne disa services, Gateway ben routing, frontend eshte me React dhe Tailwind, user mund te regjistrohet, te kyçet, te krijojë subjects/topics, te shtojë study sessions dhe quiz results, te shohe progress summary dhe te gjenerojë recommendations.

Pjesa me e rendesishme eshte se sistemi demonstroi jo vetem CRUD, por edhe business logic: total study minutes, hours, quizzes, average score, weak areas dhe recommendations. Testimi final tregoi se llogaritjet jane te sakta dhe rrjedha e sistemit funksionon nga fillimi deri ne fund.

32. Shpjegim shume i shkurter per ta thene me goje

AI Study Coach PRO is a Service-Oriented Architecture project that helps students track their study progress. The system is divided into UserService, SubjectService, StudyService, RecommendationService and ApiGateway. The frontend is built with React and communicates only with the API Gateway. The user can register, login, create subjects and topics, add study sessions and quiz results, view progress, detect weak areas and generate intelligent recommendations. The system was tested successfully with two subjects and two quiz results, calculating 105 study minutes, 1.75 hours, 2 quizzes, an average score of 55%, and one weak area.